CS 598 Project Report

Integrating Public Transit and Weather Data for Waterloo Region

1. Summary & Motivation

The objective of this project is to investigate the relationship between weather conditions and public transit operations in the Waterloo Region. Winter weather in Canada can be severe, often causing delays or service cancellations. Understanding the alignment between transit schedules and weather events is crucial for transit planners and riders.

The primary research question is: Can we create a reliable, integrated dataset that aligns planned transit service levels with historical hourly weather conditions? This project aims to support future analysis on whether extreme weather correlates with service disruptions.

The goal of this project was to build a robust, reproducible pipeline to download, assess, clean, and integrate data from two separate data sources. The final output is a high-quality, documented CSV dataset serving as a foundation for further research.

2. Dataset Profile

To achieve the project goals, there are two primary open data sources.

Grand River Transit (GRT) GTFS Data
The first dataset is the General Transit Feed Specification (GTFS) data provided by the Region of Waterloo. GTFS is the international standard for sharing public transit schedules.
- Source: Region of Waterloo Open Data Portal.
- Format: A zipped archive containing multiple text files (CSV format) such as “agency.txt”, “stops.txt”, “routes.txt”, “trips.txt”, “stop_times.txt”, and “calendar_dates.txt”.
- Reason for choosing this: It provides a relational model of the transit network. The most critical file for this project was “stop_times.txt”, which lists the arrival and departure times for every stop on every trip.
- Volume: The “stop_times.txt” file alone contains hundreds of thousands of rows, representing the schedule of the network.

Historical Weather Data
The second dataset is the historical weather data provided by Environment and Climate Change Canada (ECCC).
- Source: ECCC Historical Climate Data website.
- Station: I selected "Waterloo Int'l A" (Station ID 48569) because it is the primary weather station serving the Kitchener-Waterloo area and has consistent hourly records.
- Format: Comma-Separated Values (CSV).
- Reason for choosing this: The dataset provides hourly observations. The most relevant

columns for this project are “Date/Time (LST)”, “Temp (°C)” (Temperature), and “Precip. Amount (mm)” (Precipitation).
- Coverage: The project focused on data from 2024 to 2025 to ensure recent relevance.

3. Workflow

The curation workflow is implemented using the Python programming language. I chose Python because of its strong libraries for data manipulation (pandas) and its ability to automate file handling. The workflow was designed to be fully automated and reproducible. Any user can run the scripts and generate the exact same results.

Step 1: Data Acquisition
The first step is to automate the collection of data. I developed a script named “scripts/acquire_data.py”. Instead of manually downloading files from websites, which is prone to error, this script programmatically fetches the data.
- It downloads the latest GTFS zip file directly from the Region of Waterloo's permanent URL.
- It iterates through the months of 2024 and 2025 to download the corresponding weather data files from the ECCC API.
This ensures that we are always working with the raw, official data as published by the source.

Step 2: Data Assessment
Before using the data, it is necessary to check its quality. I created “scripts/assess_quality.py” to perform this validation.
- GTFS Assessment: The script checks to make sure all mandatory files exist and are defined by the GTFS standard. It also verifies that critical columns like “trip_id” and “arrival_time” are present.
- Weather Assessment: The script analyzes the continuity of the time series. It checks if there are any missing hours in the sequence. It also calculates the percentage of missing values for key variables. I found that approximately 0.3% of temperature records were missing, which is a small but significant gap that needs addressing.

Step 3: Data Cleaning
Based on the assessment findings, “scripts/clean_data.py” is ready to prepare the data for integration.
- Handling Missing Weather Data: To fix the missing temperature values, here I choose linear interpolation to address the gap. This assumes that the temperature changes gradually between hours. For missing precipitation values, treat them with 0. This is a reasonable assumption because precipitation is often recorded as "null" when there is no rain.
- Standardization: The script reindexes the weather data frame to ensure a perfect hourly frequency, filling any gaps in the timeline.

Step 4: Data Integration
This is the most complex phase, implemented in “scripts/integrate_data.py”.
- Aggregation: The GTFS data is event-based. It records specific times when a bus arrives at

a stop. The weather data is time-based, recorded at the top of every hour. To align them, the transit data has to be aggregated. The script calculates the "Transit Activity Count," which is the total number of scheduled stops across the entire network for each hour.
- Solving the Date Mismatch: During development, I found a significant issue. The provided GTFS data only contained a schedule for December 2025, while my weather data was for 2024. To solve this semantic heterogeneity, I implemented a data transformation rule. By shifting the GTFS dates back by exactly one year (subtracting 1 year from the date), the script can simulate an overlap between the transit schedule and the 2024 weather data.
- Merging: Eventually, two datasets are joined using “Date” and “Hour” as the common keys.

Step 5: Verification
After integration, I verified the result. I found that due to the limited date range of the GTFS source (only 3 weeks in December), the majority of the dataset had 0 transit counts. To ensure the final dataset was high quality, I filtered it to include only the dates where valid transit data existed (Dec 1-22, 2024).

4. Analysis of Lifecycle Models (M1)

My project workflow closely aligns with the DCC Curation Lifecycle Model. This model emphasizes that data curation is a continuous process, not a one-time event.

- Conceptualize: The project began by defining the research question (linking weather and transit) and identifying the necessary data sources.
- Curate and Preserve: The core of the project — the scripts for cleaning and integration, represents the curation action. By saving the final output in a non-proprietary format (CSV) and documenting the metadata, the data can thus be preserved for future use.
- Access and Use: The final dataset is structured to be easily accessible. By providing the data dictionary, it is ensured that future users can understand and use the data without needing to contact me.
- Transform: The date shift fix I implemented is a perfect example of the "Transform" action in the lifecycle, where data is modified to create new value or fit a new purpose.

5. Connection to Course Concepts

5.1 Ethical, Legal, and Policy Constraints (M2)
Conforming legal constraints is a key part of curation. Both datasets used in this project are released under the “Open Government Licence – Canada”. This license is very permissive. It allows users to copy, modify, publish, and distribute the data for both commercial and non-commercial purposes. The only major obligation is to acknowledge the source. I addressed this by explicitly listing the Region of Waterloo and Environment and Climate Change Canada as the sources in the “metadata.json” file. Since the data concerns public transit schedules and weather, there is no Personally Identifiable Information (PII) involved. Therefore, there were no ethical concerns regarding privacy or consent.

5.2 Data Models and Abstractions (M3-5)

The project required mapping between two different data models.

- Relational Model: The GTFS data uses a relational model. It consists of multiple tables (“stops”, “trips”, “times”) that are linked by foreign keys (“stop_id”, “trip_id”). This is efficient for storage but hard to analyze directly.
- Time-Series Model: The weather data is a flat time-series model, where each row represents a specific point in time.

My integration process involved abstracting the relational GTFS data. I flattened the complex relationships into a single metric (Transit_Count) per hour. This abstraction simplifies the data, making it compatible with the weather model, but it trades off some detail (e.g., we lose information about specific routes).

5.3 Data Integration and Cleaning (M6)

This module was central to my project. I encountered two main types of heterogeneity:

- Syntax Heterogeneity: One source was a zip of plain text files; the other was a collection of CSVs. I resolved this by writing parsers for each format.
- Semantic Heterogeneity: The definition of "time" was different. GTFS uses times like "25:00:00" to represent 1:00 AM on the next day for service continuity. Weather data uses standard ISO timestamps. I had to write logic to convert the GTFS "service day" time into standard calendar time to ensure accurate merging.

5.4 Metadata and Documentation (M8)

I followed the Schema.org/Dataset standard for my metadata. I chose this standard because it is widely used by search engines like Google. Using such standard makes the dataset discoverable on the web.

- MetaData: I created a “metadata.json” file using JSON-LD format. It includes fields like “temporalCoverage”, “spatialCoverage”, “creator”, and “license”.
- Data Dictionary: I also created a “data_dictionary.md”. This explains the meaning, data type, and unit of measurement for every column in the final CSV.

5.5 Identifiers (M9)

In the GTFS data, “trip_id” and “service_id” are used to uniquely identify specific bus trips and service patterns. The data source itself – that specific weather station in Waterloo is also identified by a unique ID (“48569”).

In the final integrated dataset, I used a composite key of “Date” + “Hour”. This combination uniquely identifies each row in the dataset, ensuring there are no duplicates.

5.6 Workflow Automation and Reproducibility (M12)

The workflow of this project is both automatic and reproducible. Instead of using tools like Excel, which leave no trace of the steps taken, Python scripts are used. This creates a provenance trace so that anyone can look at the code and see exactly how the data was modified.

It’s also worth noting that this project uses relative file paths (e.g., “../data/raw”) in every piece of code. This ensures that the workflow is portable and can run on any device.

5.7 Dissemination (M15)

The final step is dissemination. I packaged the project into a standardized directory structure:

- “data/”: Contains subfolders for “raw”, “processed”, and “final” data.
- “scripts/”: Contains the numbered execution scripts.
- “docs/”: Contains all documentation.

This self-contained package allows the project to be archived as a zip file or hosted on GitHub. It ensures that the data and the context required to understand it are kept together.

6. Findings, Problems, and Lessons Learned

6.1 Findings

The most significant finding was the data sparsity issue in the GTFS feed. I initially assumed the feed would contain a full year of schedule data. However, upon inspection, I found it only contained data for a 3-week period in December. Actually, this is a common characteristic of GTFS feeds, which are often published for short service periods.

6.2 Problems Encountered

The date mismatch (2025 vs 2024) was a critical problem. If I had simply tried to merge the data based on the raw dates, the result would have been an empty dataset. The date shift solution was a necessary intervention to produce a usable dataset for and only for demonstration purposes at this time.

In a more formal setup, all data should be continuously collected on a monthly base, and further processed in the following calendar month. This way, this project would function as expected based on real data, and thus better facilitates the study.

6.3 Lessons Learned

I learned that full data collection & assessment should happen before writing any integration code. If I had checked the date range of the “calendar_dates.txt” file earlier, I would have anticipated the mismatch sooner, and at least have data available for another month or more.

7. References

Environment and Climate Change Canada. (2024). “Historical Climate Data”. Retrieved from https://climate.weather.gc.ca/

Region of Waterloo. (2024). “Grand River Transit GTFS Data”. Open Data Portal. Retrieved from https://regionofwaterloo.ca/

Schema.org. (2024). “Dataset”. Retrieved from https://schema.org/Dataset

Google. (2016). “General Transit Feed Specification Reference”. Retrieved from https://developers.google.com/transit/gtfs